

Stoquify skills.sh Skill Selection

Research date: 2026-08-06 **Decision:** Adapt `playwright-best-practices` by Currents.dev **Confidence:** High for fit, medium-high for supply-chain safety until every bundled reference is manually reviewed and a commit is pinned **Execution status:** Research only. No external skill was installed or executed.

1. Executive Recommendation

The best current `skills.sh` match for Stoquify is `playwright-best-practices`, sourced from `currents-dev/playwright-best-practices-skill` at `playwright-best-practices/SKILL.md`.

Disposition: `adapt`

The candidate scores **93/100**. It is the strongest fit because Stoquify already has extensive domain, architecture, security, controls, and release skills, but its agent suite reports production readiness as `NOT_TESTED`, only 12 TypeScript/JavaScript files currently exist under `tests/`, and the agent definition suite has 999 designed evaluation cases that still need executable runtime evidence. The candidate adds detailed, progressive-disclosure Playwright guidance without attempting to replace Stoquify's server-owned business rules or domain controls.

The skill is especially well aligned with verified Stoquify test needs:

- Next.js and React browser behavior.
- Authenticated storage-state projects and RBAC denial tests.
- Multi-user and concurrent action testing.
- English/French localization and responsive testing.
- Accessibility, screenshots, visual regression, and console-error evidence.
- Offline mode, service workers, network failure, and provider degradation.
- Payment and third-party integration mocking.
- CI/CD, sharding, test isolation, traces, videos, screenshots, and reports.

Do not install it globally or use it unchanged as a release authority. Create a pinned Stoquify wrapper that routes to selected upstream references while preserving existing tenant, RBAC, financial-control, statutory, evidence, and release gates.

2. Repository Truth

Verified stack and scale

Evidence	Verified fact
<code>package.json:2</code>	The package is named <code>STOQUIFY</code> .
<code>package.json:20-21</code>	The repository has scoped linting and TypeScript type checking.
<code>package.json:22-28</code>	Prisma validation, generation, migration status, and production migration-safety gates are first-class workflows.
<code>package.json:32-74</code>	The repository has extensive agent-runtime, credential, evidence, promotion, and workflow-assurance gates.
<code>package.json:83-86</code>	Composed policy, repository, CI, and release verification commands already exist.
<code>package.json:87-110</code>	Jest and Playwright are established test frameworks.
<code>Parsed package.json</code>	Verified versions include <code>Next.js ^15.1.4</code> , <code>React ^19.1.1</code> , <code>TypeScript ^5</code> , <code>Prisma 6.19.3</code> , <code>Better Auth ^1.6.14</code> , <code>React Query ^5.76.0</code> , <code>Jest ^30.4.1</code> , <code>Playwright ^1.61.1</code> , and <code>next-intl ^4.12.0</code> .
<code>prisma/schema.prisma:31-38</code>	Prisma Client targets PostgreSQL through <code>DATABASE_URL</code> .
<code>Parsed prisma/schema.prisma</code>	The schema contains 158 models and 164 enums.
<code>prisma/schema.prisma:1-19</code>	The schema explicitly addresses tenancy, bilingual data, soft deletion, optimistic locking, payment idempotency, and per-organization uniqueness.
Source inventory	The inspected tree contains 279 app files, 226 action files, 617 service files, 312 component files, 44 hook files, 70 library files, 229 script files, and 12 files under <code>tests/</code> .

Verified business and trust domains

Representative schema models establish that Stoquify is not a generic dashboard application:

Domain	Representative evidence
Multi-tenancy and identity	<code>Organization</code> at <code>prisma/schema.prisma:237</code> , <code>User</code> at <code>:107</code> , and organization-first schema guidance at <code>:3-15</code> .
Inventory and POS	<code>Item</code> at <code>:736</code> , <code>InventoryLevel</code> at <code>:840</code> , <code>InventoryTransaction</code> at <code>:876</code> , and <code>SalesOrder</code> at <code>:3105</code> .
Purchasing and AP	<code>Supplier</code> at <code>:1012</code> , <code>PurchaseOrder</code> at <code>:1127</code> , <code>SupplierInvoice</code> at <code>:1414</code> , and <code>SupplierPayment</code> at <code>:1545</code> .
Payroll and HR	<code>PayrollEmployee</code> at <code>:1800</code> , <code>PayrollRun</code> at <code>:2650</code> , <code>PayrollPayslip</code> at <code>:2769</code> , and <code>PayrollDeclaration</code> at <code>:2848</code> .
Payments and reconciliation	<code>Payment</code> at <code>:4081</code> , <code>PaymentTransaction</code> at <code>:4671</code> , <code>ReconciliationRun</code> at <code>:4837</code> , and <code>PaymentException</code> at <code>:4896</code> .
Accounting and close	<code>AccountingPeriod</code> at <code>:5216</code> , <code>Journal</code> at <code>:5732</code> , <code>LedgerPostingBatch</code> at <code>:5867</code> , and <code>LedgerAuditEvent</code> at <code>:6018</code> .
Events and compliance	<code>BusinessEvent</code> at <code>:6104</code> , <code>BusinessEventOutbox</code> at <code>:6152</code> , <code>ComplianceSubmission</code> at <code>:6397</code> , and <code>ComplianceEvidence</code> at <code>:6505</code> .
Auditability	<code>AuditLog</code> at <code>prisma/schema.prisma:7079</code> .

Verified test architecture

`playwright.config.ts` already provides a mature starting point:

- `playwright.config.ts:20-40` configures the E2E directory, CI retries and reporters, trace retention, screenshots, video, and managed web-server startup.

- `playwright.config.ts:61-177` defines authentication setup and dedicated projects for payroll, command-agent desktop/mobile/degradation, HRIS, close assurance, negative RBAC, transaction history, and inventory loss.
- `tests/e2e/command-agent-enabled-pilot.spec.ts:12-77` verifies bounded command-agent behavior, keyboard operation, evidence links, accessibility, idempotent retry behavior, and responsive overflow.
- `tests/e2e/command-agent-enabled-pilot.spec.ts:79-110` verifies denial for a role outside the pilot allowlist.
- `tests/e2e/close-assurance-authenticated-smoke.spec.ts:101-181` verifies authenticated close workflows, evidence screenshots, controlled exports, hashes, draft-only certification semantics, and redaction.

The repository therefore needs deeper test architecture and coverage guidance, not a replacement framework.

Verified agent and skill readiness gap

- `docs/copilot/stoquify-agent-skill-definition-suite/reports/completion-report.md:3-6` reports source creation complete but production readiness not tested.
- `docs/copilot/stoquify-agent-skill-definition-suite/reports/validation-report.md:3-10` reports structural validation passing for 9 agents, 28 skills, 37 registry records, and 999 designed evaluation cases.
- `docs/copilot/stoquify-agent-skill-definition-suite/reports/validation-report.md:16-18` explicitly states runtime behavior, integrations, load, canary operation, suspension, and rollback remain untested.
- `docs/copilot/stoquify-agent-skill-definition-suite/reports/post-installation-next-steps.md:20-24` calls for converting the 999 designed cases into executable fixtures with tenant, role, module, location, country-pack, offline, provider-timeout, approval, and prompt-injection datasets.
- `what-next/STOQUIFY_AGENT_RUNTIME_EXTERNAL_INPUT_READINESS_2026-08-03.md:3-9` reports `EXTERNAL_INPUTS_REQUIRED`, 1 of 13 checks passed, 102 blockers, and no activation or Phase 3 authority.
- `what-next/STOQUIFY_AGENT_RUNTIME_EXTERNAL_INPUT_READINESS_2026-08-03.md:46-54` requires immutable release identity, distinct approvals, evidence channels, statutory review, and explicit GO before pilot activation.

This evidence makes runtime assurance the highest-value cross-platform capability gap. External deployment identities, approvals, secrets, and statutory review remain organizational blockers and cannot be solved by any downloaded skill.

Graph evidence

The required search for `graphify-out/GRAPH_REPORT_*.md` returned no reports. Architecture claims in this assessment therefore rely on repository files, schema evidence, source layout, module inventory, and existing reports. Graph-based dependency/community findings are marked unavailable rather than inferred.

3. Existing Skill Coverage and Overlap

The installed skill inventory contains hundreds of Stoquify, AqStoqFlow, Kontava, StockFlow, security, architecture, Azure, UI/UX, accounting, POS, inventory, payroll, module-control, workflow-assurance, and release-gate skills. The main selection risk is duplication and conflicting instructions.

Relevant installed skills

Installed skill	Existing ownership	Gap left for the candidate
aqstoqflow-uiux-09-accessibility-visual-regression-governance	Defines high-value route, accessibility, responsive, screenshot, route-maturity, and UI-governance outcomes. See its SKILL.md:35-50.	It intentionally gives only high-level guidance and says to use existing test tooling. It does not provide a broad Playwright pattern library.
aqstoqflow-release-verification-foundation	Defines authenticated smoke routes, migration checks, build evidence, and truthful stop conditions. See its SKILL.md:18-53.	It does not provide detailed fixtures, selectors, flake control, multi-user contexts, network simulation, test architecture, or CI scaling guidance.
aqstoqflow-workflow-assurance-observe-pilot	Owns observe-mode incident evidence and promotion gates. See its SKILL.md:12-34.	It is a business assurance workflow, not a browser-test engineering reference.
stoquify-pos-chaos-field-certifier	Owns POS database, offline, provider, browser, hardware, performance, and recovery certification. See its SKILL.md:8-49.	It is POS-specific and certifies immutable candidates. It does not establish reusable Playwright engineering patterns for every module.
review	Provides evidence-backed, severity-ranked code review.	It reviews changes but is not a detailed Playwright reference.
SkillsSmith	Splits, merges, sequences, and audits overlapping local skills.	It manages skill design and boundaries, not application E2E implementation.
find-skills	Searches skills.sh and offers installation.	Its quality guidance overweights install count and does not perform Stoquify-specific architecture, security, overlap, or provenance analysis.

Net-new value

playwright-best-practices is complementary because it supplies reusable implementation mechanics beneath Stoquify-owned outcomes. The local skills remain authoritative for what must be proven; the external skill can help engineers choose how to implement Playwright evidence safely.

4. Search Strategy

The live skills.sh catalog was searched on 2026-08-06. Search terms were derived from the verified repository stack and gaps.

Queries used:

- site:skills.sh Next.js React TypeScript Prisma agent skill
- site:skills.sh security code review agent skill
- site:skills.sh PostgreSQL Prisma database agent skill
- site:skills.sh agent skill evaluation skill creator audit
- site:skills.sh React best practices Vercel agent skill
- site:skills.sh Next.js best practices agent skill Vercel
- site:skills.sh Playwright webapp testing agent skill
- site:skills.sh evals agent evaluation testing skill
- site:skills.sh/obra/superpowers verification before completion
- site:skills.sh/obra/superpowers test driven development
- site:skills.sh observability SRE release readiness skill
- site:skills.sh multi tenant SaaS RBAC Next.js skill

- `site:skills.sh playwright-best-practices`
- `site:skills.sh skill-evaluator agent skills`

Search results were not treated as sufficient evidence. Serious candidates were opened on `skills.sh`, and their linked GitHub repositories or exact skill files were inspected where available.

5. Ranked Shortlist

Scoring uses the required 100-point rubric.

Rank	Candidate	Stack fit /20	Gap /20	Instructions /15	Security /15	Domain /10	Compatibility /10	Maintenance /5	Effort /5	Total	Disposition
1	playwright-best-practices	20	18	15	13	9	9	5	4	93	adapt
2	vercel-react-best-practices	19	10	15	14	5	8	5	5	81	adopt for focused performance work
3	agent-skills-creator	8	7	14	14	5	3	4	3	58	inspiration only

1. playwright-best-practices

- **Author:** Currents.dev.
- **skills.sh:** [candidate page](#).
- **Source:** [repository and exact skill directory](#).
- **Exact path:** `playwright-best-practices/SKILL.md`.
- **Claimed purpose:** Comprehensive TypeScript Playwright guidance for E2E, component, API, visual, accessibility, security, framework, debugging, and CI/CD testing.
- **Verified source shape:** One `SKILL.md` plus reference directories for advanced, architecture, browser APIs, core, debugging, frameworks, infrastructure/CI, and test patterns. No executable script directory is present in the inspected skill root.
- **License:** MIT.
- **Catalog signals at research time:** Approximately 68.8K installs, 348 GitHub stars, first seen 2026-01-29, and Pass results for Gen Agent Trust Hub, Socket, and Snyk.
- **Stoquify use cases:** Auth/RBAC contexts, multi-user/maker-checker flows, mobile and desktop projects, EN/FR, accessibility, offline replay, network/provider failures, payment mocking, screenshots, traces, evidence artifacts, CI sharding, and flake reduction.
- **Overlap:** Moderate with UI/UX governance, release verification, workflow assurance, and POS chaos skills, but those local skills own outcomes and controls rather than broad Playwright mechanics.
- **Dependencies:** Assumes Playwright and Node tooling. Stoquify already has `@playwright/test` and an established TypeScript Playwright configuration.
- **Permissions:** The skill is instructional. Applying it may run project-defined test commands and write test artifacts, but it does not itself require credentials or elevated privileges.
- **Risk:** The 57 referenced documents were not individually security-audited in this run. Pin and review the selected upstream commit before adoption. Do not allow generic examples to weaken tenant, RBAC, redaction, or evidence requirements.
- **Customization effort:** Medium. No framework installation is needed, but a wrapper should constrain triggers and map references to Stoquify gates and evidence locations.

2. vercel-react-best-practices

- **Author:** Vercel.
- **skills.sh:** [candidate page](#).
- **Source:** [exact SKILL.md](#).
- **Exact path:** `skills/react-best-practices/SKILL.md`.
- **Claimed purpose:** Seventy React and Next.js performance rules covering waterfalls, bundles, server behavior, client data fetching, rerenders, rendering, and JavaScript performance.
- **Verified fit:** Exact Next.js/React alignment, including server-action authentication, request-state isolation, data-fetch parallelism, serialization, and bundle control.
- **Catalog signals at research time:** The repository had approximately 29.8K stars and Pass results for Gen Agent Trust Hub, Socket, and Snyk. The candidate page did not expose a stable individual install count.
- **Overlap:** Moderate with Stoquify UI/UX and performance skills, but it adds a maintained framework-performance rule base.
- **Risk:** Performance advice can conflict with correctness, fresh authorization, audit evidence, or transaction ordering if applied mechanically. Stoquify service and security contracts must win every conflict.
- **Customization effort:** Low. Use it unchanged only for focused React/Next.js performance reviews, not as a platform-wide orchestrator.

3. agent-skills-creator

- **Author:** Matthew Blode / `mblode`.
- **skills.sh:** [candidate page](#).
- **Source:** [exact SKILL.md](#).
- **Exact path:** `skills/agent-skills-creator/SKILL.md`.
- **Claimed purpose:** Create and improve open-format skills with references, scripts, evaluations, validation, rightsizing, and an eleven-dimension audit.
- **Catalog signals at research time:** Approximately 599 installs, 75 repository stars, 242 repository commits, MIT license, and Pass results for all three listed security audits.
- **Dependencies and permissions:** Uses repository scripts such as `scripts/validate.sh`, can install through `npx skills`, can create symlinks, and is designed to write skill and documentation files.
- **Overlap:** High. Installed `SkillSmith`, the system `skill-creator`, the Stoquify prompt architect, and the 28-skill definition suite already cover the same lifecycle with more project context.
- **Risk:** A broad audit or rewrite could create instruction conflicts or erase deliberate Stoquify domain constraints if used without strict boundaries.
- **Customization effort:** Medium. Only borrow its constraint-ablation and cross-model evaluation ideas through `SkillSmith`; do not add another competing creator skill.

6. Rejected and Watchlist Candidates

Candidate	Decision	Reason
agent-evaluation	Reject for now	Strong conceptual match to the 999-case gap, but the linked GitHub repository returned 404 during inspection and Gen Agent Trust Hub showed Warn. Source provenance is insufficient.
skill-evaluator	Watchlist	The skill labels itself WIP, has about 60 installs, and largely overlaps local structural validation and SkillSmith. The parent SkillPort project is interesting for search-first loading across very large skill collections, but it adds Python, CLI/MCP, installation, update, and removal capabilities that need a separate security and operations decision.
verification-before-completion	Inspiration only	Its evidence-first principle is sound, but Stoquify already encodes stronger domain-specific policy, repository, CI, release, and evidence gates in package.json.
test-driven-development	Reject as a global rule	Its absolute requirement to delete production code written before tests conflicts with surgical brownfield work, dirty-worktree safety, and risk-proportional verification.
prisma-cli	Reject for current gap	Official and well-maintained, but focused on Prisma 7 CLI guidance while Stoquify is pinned to Prisma 6.19.3 and already has extensive migration safety gates.
prisma-database-setup	Reject for current gap	Stoquify's PostgreSQL datasource and Prisma client are already established; setup guidance does not address the demonstrated readiness gap.
secure-code-review	Reject	The ASVS/CWE structure is useful, but the catalog showed a Snyk failure, low adoption, and significant overlap with installed security scanning and RBAC skills.
nextjs-saas	Reject	Assumes Next.js 16+ and Supabase auth, while Stoquify uses Next.js 15.1.4 and Better Auth with mature custom tenancy and RBAC boundaries.

7. Security and Supply-Chain Assessment

Winner

Control	Assessment
Inspectable source	Pass. The repository, skill directory, SKILL.md, reference structure, history, and license were visible.
Executable scripts	Pass for inspected root. No script directory was present in the candidate skill root.
Credentials	Pass. No credential request was found in the inspected skill instructions.
Elevated permissions	Pass. No elevation requirement was found.
Destructive commands	Pass. The inspected skill instructions focus on test commands and iterative reruns.
External network	Not required by the skill instructions for normal use, beyond acquiring the package if installation is later authorized.
Telemetry	Unknown for the skill itself. The skills CLI documentation states that install telemetry is collected by default unless DISABLE_TELEMETRY=1 is set.
Catalog audits	Pass for Gen Agent Trust Hub, Socket, and Snyk at research time. These are signals, not certification.
Residual risk	Medium-low before pinning, medium if installed from a moving branch. Review all selected references and pin an immutable commit.

Required adoption controls

- Do not install from an unpinned moving branch in a production-capable agent environment.
- Review the complete candidate directory at the selected commit.
- Record repository, commit SHA, tree hash, license, reviewer, and review date.
- Vendor or wrap only the needed references; avoid loading all 57 documents for every test task.
- Keep upstream commands subordinate to Stoquify's existing package scripts and release gates.
- Preserve server-derived tenancy, RBAC, module entitlement, fresh authentication, maker-checker, redaction, and audit controls.
- Never let browser success certify ledger correctness, statutory compliance, provider settlement, migration safety, or production readiness by itself.
- Execute external network, provider, email, SMS, or payment scenarios only in controlled test environments with synthetic data.

8. Proposed Adaptation

Proposed local wrapper

Name: stoquify-playwright-assurance

Purpose: Route Stoquify browser, accessibility, localization, offline, provider-failure, multi-user, and release-evidence work to reviewed Playwright references while preserving Stoquify's domain and security gates.

Trigger conditions:

- Adding or reviewing Playwright E2E, browser-smoke, accessibility, screenshot, responsive, security, or performance tests.
- Debugging flaky authenticated flows, test data leakage, storage-state errors, timeouts, race conditions, or CI-only failures.
- Testing tenant/RBAC denial, maker-checker, multi-user, offline replay, provider timeout, payment, export/redaction, or EN/FR behavior.
- Designing fixtures, project dependencies, test tagging, CI sharding, trace collection, or evidence artifacts.

Required repository context:

- `playwright.config.ts`
- Relevant `tests/e2e/*.spec.ts`
- Applicable package scripts and policy/release gates
- Domain service and action contracts
- Tenant, RBAC, module entitlement, redaction, and evidence requirements
- Relevant local domain skill, such as POS chaos, close assurance, payroll, reconciliation, or UI/UX governance

Operating boundaries:

- Upstream Playwright references own test mechanics only.
- Stoquify services own business truth.
- Local domain skills own invariants and acceptance criteria.

- Local release gates own promotion decisions.
- No live payment, payroll, statutory filing, ledger posting, write-off, certification, or destructive database action may be triggered by a browser test.

Outputs:

- Focused tests and fixtures when implementation is authorized.
- Exact commands and results.
- Trace, screenshot, video, console, download, and hash evidence where applicable.
- Explicit `PASS`, `FAIL`, `BLOCKED`, or `NOT RUN` per scenario.
- Residual risks and release-gate handoff.

Validation approach:

- Validate the wrapper structurally with the existing skill validator.
- Test trigger precision against unrelated backend and documentation prompts.
- Run baseline prompts without the wrapper and compare them with wrapper-enabled outcomes.
- Pilot on command-agent RBAC, close-assurance export/redaction, and one offline POS/provider-degradation flow.
- Require repeat runs for critical flows and preserve trace evidence for failures.
- Do not claim production readiness until external identities, approvals, evidence channels, and statutory review are complete.

9. Adoption Plan

- 1 Obtain explicit approval before downloading or installing anything.
- 2 Resolve and pin an immutable Currents.dev repository commit.
- 3 Review every reference selected for Stoquify use and record a provenance manifest.
- 4 Create the narrow `stoquify-playwright-assurance` wrapper instead of globally replacing local test and release skills.
- 5 Map upstream references to existing Playwright projects and package commands.
- 6 Pilot only on existing read-only or controlled test flows.
- 7 Compare flake rate, failure diagnosis time, evidence completeness, accessibility coverage, RBAC denial coverage, and CI duration before and after the pilot.
- 8 Promote the wrapper only if it improves evidence quality without weakening local controls or creating instruction conflicts.

Estimated customization effort: **Medium**.

No new runtime dependency is expected because Playwright is already installed. The work is primarily provenance review, trigger design, reference selection, local-control mapping, and evaluation.

10. Reviewer Lens Decisions

Lens	Decision
Enterprise/platform architecture	Applicable. The candidate must remain a test-mechanics layer and not become a domain orchestrator.
Backend/API/distributed systems	Applicable where browser tests cover API, provider, concurrency, or failure behavior; server contracts remain authoritative.
Database/migration	Applicable only for fixture isolation and environment setup. The candidate must not own migration decisions.
Security/IAM/RBAC/privacy	Applicable and high priority because the candidate supports auth, multi-user, security, and denial tests.
Frontend/design system	Applicable through Next.js, React, responsive, accessibility, and visual testing.
Workflow/service UX	Applicable through end-to-end flows, error states, keyboard behavior, and recoverability.
Product/business process	Applicable for scenario selection and outcome coverage, not for product strategy decisions.
Finance/accounting/reconciliation	Applicable as scenario domains; browser evidence cannot certify accounting truth.
OHADA/statutory/country packs	Applicable for bilingual and workflow evidence only; legal/statutory certification remains human-owned.
Quality/release assurance	Primary fit.
SRE/DevSecOps/observability	Applicable through CI, reports, traces, failure artifacts, performance, and flake control.
Offline/POS/provider boundaries	Applicable through service-worker, offline, network, third-party, and multi-context testing.
Analytics/data governance	Not applicable to skill selection beyond protecting test data and evidence retention.
AI/agent safety	Applicable through command-agent E2E, denial, evidence, kill-switch, and degradation scenarios.
Packaging/billing/growth/customer success	Not applicable to the candidate's core function; use it only when those modules need browser assurance.

11. Verification Results

Check	Status	Evidence
Live <code>skills.sh</code> searched with repository-derived terms	Passed	Search performed on 2026-08-06 across framework, database, security, testing, skill lifecycle, SaaS, RBAC, and evaluation terms.
Existing skill overlap evaluated	Passed	Installed inventory and five general skills plus four nearest Stoquify assurance skills were inspected.
Winner inspected beyond search summary	Passed	<code>skills.sh</code> , GitHub repository, exact skill directory, complete <code>SKILL.md</code> , repository structure, license, and catalog audits were inspected.
Winner scripts reviewed	Passed	No executable scripts were present in the inspected skill root.
Dependencies and permissions documented	Passed	Existing Playwright/Node dependency confirmed; no credentials or elevation required by the inspected instructions.
Scores supported by evidence	Passed	Rubric breakdown and candidate-specific evidence are recorded above.
Adoption boundaries and risk controls defined	Passed	Sections 7 through 9.
Graph architecture reports inspected	Blocked	No <code>graphify-out/GRAPH_REPORT_*.md</code> reports were returned by the repository inventory.
External skill installed	Not applicable	Installation was explicitly outside scope and was not performed.
External skill executed	Not applicable	Candidate instructions and scripts were not executed.
Production readiness certified	Not applicable	Existing reports explicitly state production readiness is not tested and external inputs remain blocked.

12. Final Decision

Best Overall Match: `playwright-best-practices` by Currents.dev **Decision:** Adapt into a pinned `stoquify-playwright-assurance` wrapper **Score:** 93/100 **Confidence:** High for functional fit; medium-high for supply-chain confidence pending full reference review and commit pinning

It is better suited to Stoquify than the alternatives because it addresses a demonstrated cross-platform runtime-evidence gap, matches the existing TypeScript Playwright stack, covers the platform's hardest browser scenarios, and complements rather than competes with Stoquify's extensive domain-specific skill suite.

It should not be used as a substitute for service-layer tests, database constraints, financial invariants, security review, statutory review, provider reconciliation, operational approvals, or release certification.

Sources

- [skills.sh documentation and security caveat](#)
- [playwright-best-practices on skills.sh](#)
- [playwright-best-practices source](#)
- [Vercel React Best Practices on skills.sh](#)
- [Vercel React Best Practices source](#)
- [Agent Skills Creator on skills.sh](#)
- [Agent Skills Creator source](#)

- [Skill Evaluator on skills.sh](#)
- [SkillPort repository](#)
- [Agent Evaluation on skills.sh](#)
- [Verification Before Completion on skills.sh](#)
- [Test-Driven Development on skills.sh](#)
- [Prisma CLI on skills.sh](#)
- [Secure Code Review on skills.sh](#)
- [Next.js SaaS on skills.sh](#)